

Les types de données génériques en Rust

✍ En bref...

- Les types génériques permettent de définir des [structures](#), [fonctions](#) et traits pouvant fonctionner avec différents types de données sans duplication de code.
- La syntaxe `<T>` déclare un paramètre de type générique, utilisé dans les fonctions, structures, [énumérations](#) et traits.
- Les traits génériques définissent des comportements communs à plusieurs types, favorisant la réutilisation et la sécurité de type.
- Les énumérations génériques, comme `Option<T>`, permettent de modéliser des données optionnelles indépendamment du type.

Introduction

Les types de données génériques sont une pierre angulaire de la programmation [Rust](#), offrant une puissance expressive pour créer du code flexible, réutilisable et sûr. Ils permettent d'écrire des fonctions, structures, traits et énumérations qui fonctionnent avec divers types de données, tout en garantissant la sécurité de type à la compilation.

Concepts fondamentaux des types génériques

Définition et utilité

Les types génériques permettent de paramétrer des définitions de fonctions, structures, traits et énumérations par des types non spécifiés à l'avance. En utilisant la syntaxe `<T>`, on déclare un paramètre de type générique qui peut être remplacé par n'importe quel type concret lors de l'utilisation. Cela évite la duplication de code tout en conservant la sécurité de type, car le compilateur vérifie que les types utilisés respectent les contraintes définies.

Paramètres génériques

Les paramètres génériques peuvent être utilisés dans les signatures de fonctions, les définitions de structures, les énumérations et les traits. Ils permettent d'abstraire le type réel utilisé, rendant le code plus flexible et réutilisable. Par convention, les noms de paramètres génériques sont souvent courts (ex. `T`, `K`, `V`).

Sécurité de type et performance

[Rust](#) garantit que l'utilisation de types génériques ne dégrade pas la performance : le code généré est aussi efficace que si les types étaient concrets. Le compilateur génère des instances spécialisées pour chaque type utilisé, assurant ainsi une exécution rapide et une sécurité mémoire.

Structures génériques

Les structures génériques permettent de définir des types capables de stocker des données de types variés. Par exemple, une structure `Point<T>` peut contenir des coordonnées de type `T`, où `T` est un paramètre générique.

```
struct Point<T> {  
    x: T,  
    y: T,  
}
```

```

}

fn main() {
    let integer_point = Point { x: 5, y: 10 }; // T est i32 par inférence
    let float_point = Point { x: 1.0, y: 4.0 }; // T est f64 par inférence
    println!("Point i32: ({} , {})", integer_point.x, integer_point.y);
    println!("Point f64: ({} , {})", float_point.x, float_point.y);
}

```

Fonctions génériques

Les fonctions génériques acceptent des paramètres de types génériques, permettant d'écrire des algorithmes indépendants des types concrets. Par exemple, une fonction `le_plus_grand` peut comparer et retourner le plus grand élément d'une liste, quel que soit le type `T` des éléments, à condition que `T` implémente le trait [PartialOrd](#) (comparaison partielle).

```

fn le_plus_grand<T: std::cmp::PartialOrd>(list: &[T]) -> &T {
    let mut le_plus_grand = &list[0];
    for item in list {
        if item > le_plus_grand {
            le_plus_grand = item;
        }
    }
    le_plus_grand
}

fn main() {
    let numbers = vec![34, 50, 25, 100, 65];
    let chars = vec!['y', 'm', 'a', 'q'];
    println!("Le plus grand nombre est {}", le_plus_grand(&numbers));
    println!("Le plus grand caractère est {}", le_plus_grand(&chars));
}

```

Traits et implémentations génériques

Les traits définissent des comportements communs que plusieurs types peuvent implémenter. Ils permettent la surcharge d'opérateurs et la création de types génériques. Par exemple, un trait `Printable` peut définir une méthode `print` que différentes structures implémentent.

```

trait Printable {
    fn print(&self);
}

struct Book {
    name: &'static str,
    id: u32,
}

impl Printable for Book {
    fn print(&self) {
        println!("Livre : id = {}, nom = {}", self.id, self.name);
    }
}

fn main() {
    let book = Book { id: 1001, name: "Introduction à Rust" };
}

```

```
    book.print();  
}
```

Si nous prenons l'idée d'un gestionnaire bibliographique, les traits permettent de définir des comportements communs à différentes entités (livres, auteurs, bibliothèques), facilitant la maintenance et la cohérence du code.

Énumérations génériques

Les énumérations génériques permettent de définir des types pouvant contenir différentes variantes de types. L'exemple classique est `Option<T>`, qui représente une valeur optionnelle de type `T`.

```
enum Option<T> {  
    Some(T),  
    None,  
}  
  
fn main() {  
    let some_number = Option::Some(5);  
    let some_string = Option::Some("texte");  
    let none: Option<i32> = Option::None;  
    println!("{:?}, {:?}", some_number, some_string, none);  
}
```

Dans un contexte précis, nous pourrions chercher à modéliser la présence ou l'absence d'une information (par exemple, un champ optionnel dans une fiche de livre, dans le cadre d'un gestionnaire bibliographique).

Utilisation des génériques dans les collections

Les collections `Rust`, telles que `HashMap`, utilisent des génériques pour fonctionner avec différents types de clés et valeurs. Cela permet de créer des structures de données flexibles et réutilisables.

```
use std::collections::HashMap;  
  
fn main() {  
    let mut books: HashMap<u32, String> = HashMap::new();  
    books.insert(1, "Introduction à Rust".to_string());  
    books.insert(2, "Rust en Action".to_string());  
    println!("Bibliothèque : {:?}", books);  
}
```